PYTHON FOR HEALTHCARE · STANFORD TECHNOLOGY TRAINING

Check a Model Before You Trust It

Session 2 handout · Prediction

An assistant can fit six models in one loop. Choosing the model is the easy part; the evaluation is the work.

Pattern	What it does	Watch for
SPLITTING AND FITTING		
`train_test_split(X, y, test_size=0.25, stratify=y, random_state=42)`	The same train and test rows every run	Another `random_state` is another test set
`make_pipeline(SimpleImputer(), StandardScaler(), model)`	Learns fill-in values and scaling from training rows only	Filling or scaling before the split lets the test set leak in
`model.fit(X_train, y_train)`	Trains the model	Never fit on test rows
SCORING		
`model.predict_proba(X_test)[:, 1]`	Risk scores from 0 to 1	`predict()` gives 0 or 1 at a 0.5 cutoff; an AUROC from those is wrong
`roc_auc_score(y_test, risk)`	Ranking: 0.5 is a coin flip, 1.0 is perfect	Too good to be true usually means a leak
`accuracy_score(y_test, model.predict(X_test))`	Share correct at a 0.5 cutoff	"No one gets sick" can score 78%
`np.quantile(risk, 0.80)`	The cutoff for the top 20%	Capacity sets the cutoff, not 0.5
`pd.qcut(risk, 10)`	Tenths of predicted risk, for calibration	Compare predicted with observed in each tenth
INSPECTING		
`permutation_importance(model, X_test, y_test, scoring="roc_auc")`	What the model leans on	Leaks hide at the top of this list
`dictionary.loc[dictionary["recorded"] == "index visit", "column"]`	Columns known when the model is used	An allow-list refuses new columns by default
`SVC(probability=True)`	SVM probabilities (older code)	Deprecated in scikit-learn 1.9; use `CalibratedClassifierCV(SVC(), ensemble=False)`

Eight checks for any prediction model, whoever wrote the code

Check	PASS if
1. Known when it's used	Every feature was recorded before the decision the model supports. Check the dictionary, then write the rule as a refusal.
2. An honest test	Test rows never touch training, preprocessing sits inside the pipeline, and the seed is fixed.
3. Believable	The score is plausible for the question. A superb score is a reason to look harder.
4. The use-case table	Sensitivity and PPV at the program's real capacity, in patients, not just an AUROC.
5. Calibrated	Predicted and observed rates agree in each tenth of risk.
6. Somewhere else	Ranking and calibration hold at another site or time, or you know how they fail.
7. Who's missed	Results by subgroup, including missing data by group.
8. Do-not-use rules	Written down, specific, and checkable.

PYTHON FOR HEALTHCARE · STANFORD TECHNOLOGY TRAINING

Check a Simulation Before You Trust It

Session 2 handout · Operations

Pattern	What it does	Watch for
`env = simpy.Environment()`	The simulated clock	Time units are your choice (hours here)
`beds = simpy.Resource(env, capacity=7)`	A pool of identical beds or rooms	Capacity is the setting you'll change
`req = beds.request()` then `yield req`	Wait until a bed is free	The wait is the gap between asking and getting
`yield env.timeout(hours)`	Spend time: a stay	The stay's distribution is an assumption
`beds.release(req)`	Free the bed	Released only after the next bed is granted: that's blocking
`rng = np.random.default_rng(seed)`	Seeded random numbers	Same seed, same patients

Seven checks for any simulation

Check	PASS if
1. Volume	The births counted match births a day × days simulated.
2. Warm-up	The first weeks, while the empty unit fills, are left out.
3. Replications	At least 20 runs, reported as a mean with a 95% interval. One run is an anecdote.
4. A fair comparison	Every scenario runs on the same seeds, so it sees the same patients (the births row changes by +0.0).
5. The metric first	You named the deciding number before you ran the comparison.
6. Refusals	It won't report when demand reaches capacity, or when waits pass anything a real unit would allow.
7. Assumptions on paper	Arrival pattern, stay distributions, and what's left out (diversion, cesareans, NICU).

At Stanford: Fair, Useful, Reliable

Stanford Health Care evaluates AI models before they're used in care with **FURM** assessments: is it fair, useful, and reliable, and can it be sustained (Callahan et al., *NEJM Catalyst*, 2024). Lab 3 was a miniature: the capacity table (useful), Site B (reliable), and who's missed (fair). To report a prediction model, use **TRIPOD+AI** (Collins et al., *BMJ*, 2024), a 27-item checklist for regression and machine-learning models.

Real data

Synthetic data only in Colab and Gemini on a personal account. For where real data can go at Stanford, see page 2 of the Session 1 handout, and check **uit.stanford.edu/ai/matrix** first.

Go further

TRIPOD+AI checklist: tripod-statement.org · SimPy and Streamlit for health services (Monks and Harper, MIT license): github.com/health-data-science-OR/simpy-streamlit-tutorial · An introduction to DES for healthcare (HSMA): des.hsma.co.uk